Agents: Foundations & Planning

Satya Pattnaik

Agents : foundations

advance concepts

protocols — MCP

LLM → SFT ⟶ Unsloth

LoRA ⟶

DDP/FSDP

Scaling AI App
Inference optimizations } vLLMs }

Quantization

async programming

Pydantic

Dockerize the App on Pods

Agents → Langchain / Langgraph Deepdive

Agent → Multi Agent orchestration

Agent → Crew AI | Autogen

Agent
Reflection / Memory →

Agent Browser Agent

Rag → LlamaIndex

Agent + Rag → Agentic RAG + Graph RAG Neo4J

Hybrid Search + Retrieval

Parse complex doc

DSPy ⇐ Prompt

LLM as a Judge } Eval

HITL ⇐ Agent

Security / Low Code } 2 class

2 class — Diffusion Models

Goa - 4 day 50K

→ Flights

→ Hotels

Breakfast

No breakfast

→ Getyourguide (activities)

→ B/L/D

call

30K

80K

15K

1.5L

LLM

→ get flight prices

is it trained?

15K

30K

→ F → H → G → B/L/D → User

api

Hotels

	€	location	checkin	checkout
H1	$1	B		
H2	$2	NB		
H3	$3			

api

P1	P2	P3	P4	P5
from	to	date	time	premium
→ F1	$1			
→ F2	$2			
→ F3	$3			

What is count of customers? Ag queries → result

Text-to-SQL

Salesforce Developers

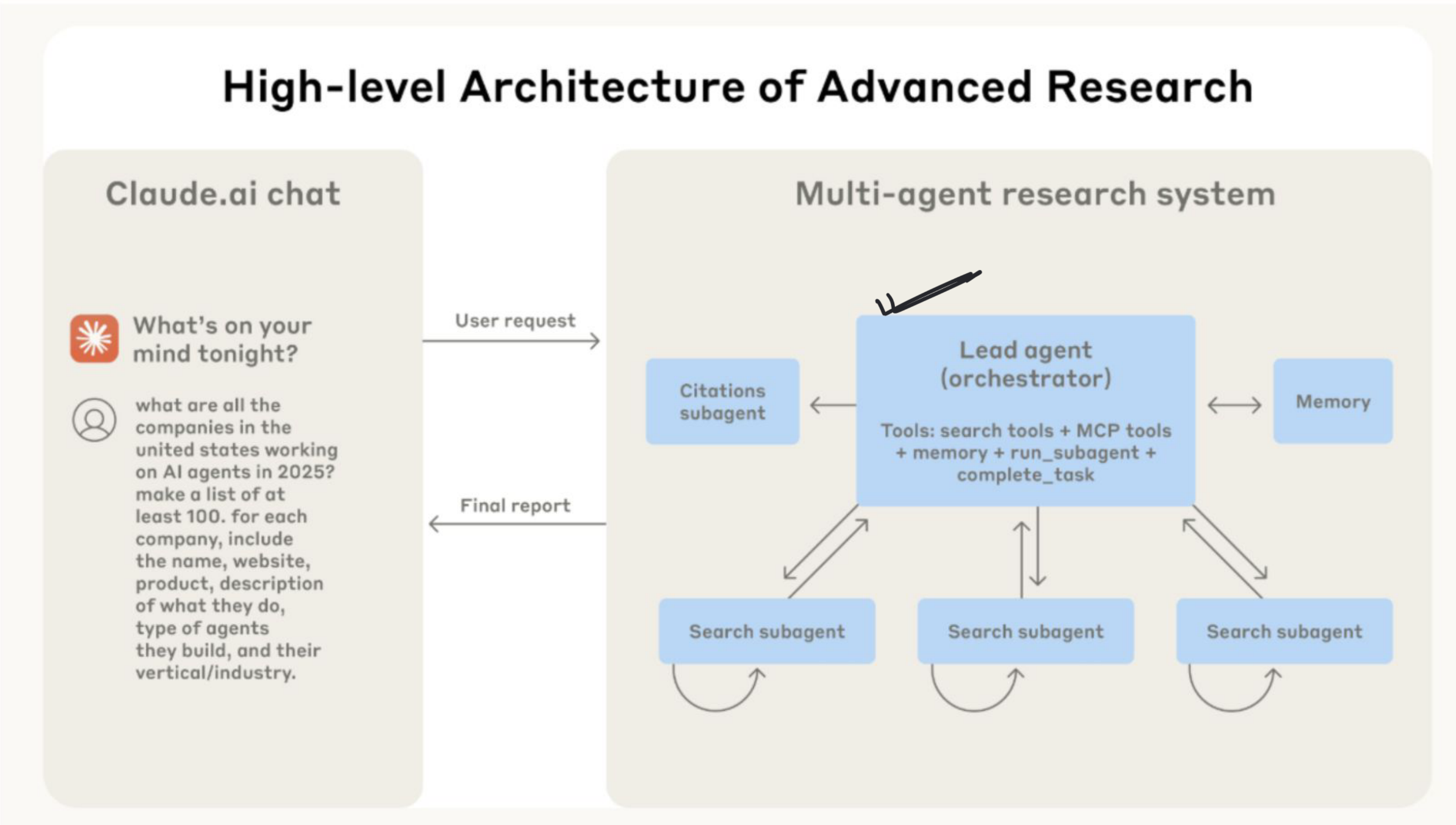
How We Built a Text-To-SQL AI Agent to Get Instant Answers From Our Data

Advancements in large language models have made AI data analysis and SQL generation easier for non-technical people. Here's how.

Our technologists used to spend dozens of hours a week working on custom queries for our users. Now users can self-serve answers from our text-to-SQL Slack agent in minutes. [Image credit: peopleimages.com / Adobe Stock]

Do research on GDP of India

Ag1 Govt Web
Ag2 Economic Papers
Ag3 Journalist articles



Data → FE → Train → HP → Eval → Deployed

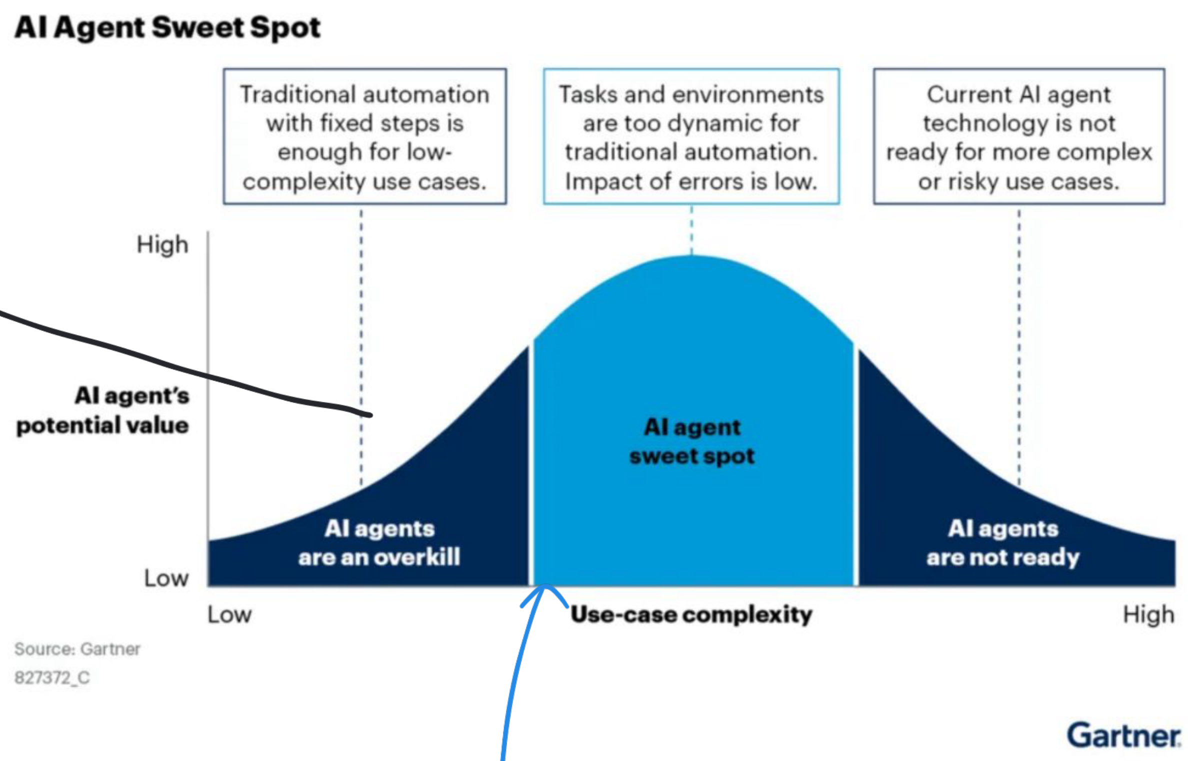
New code → Pushed → unit tests → stress tests → integration

IT ticket resolution agent

Jira Board is not working → Agent — KB / × / Web / Human in the loop

90%.

Figure 7: AI Agent Sweet Spot

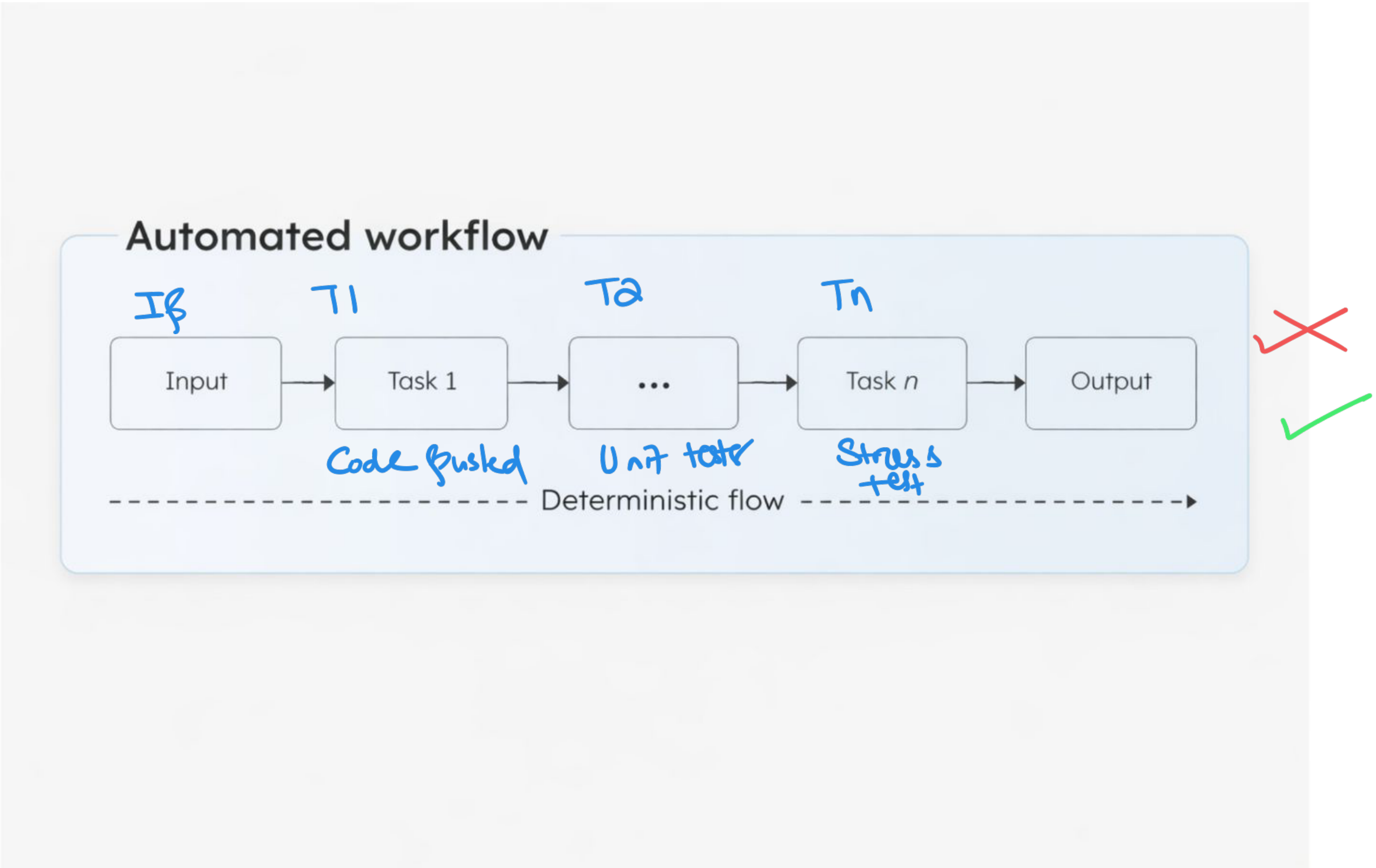


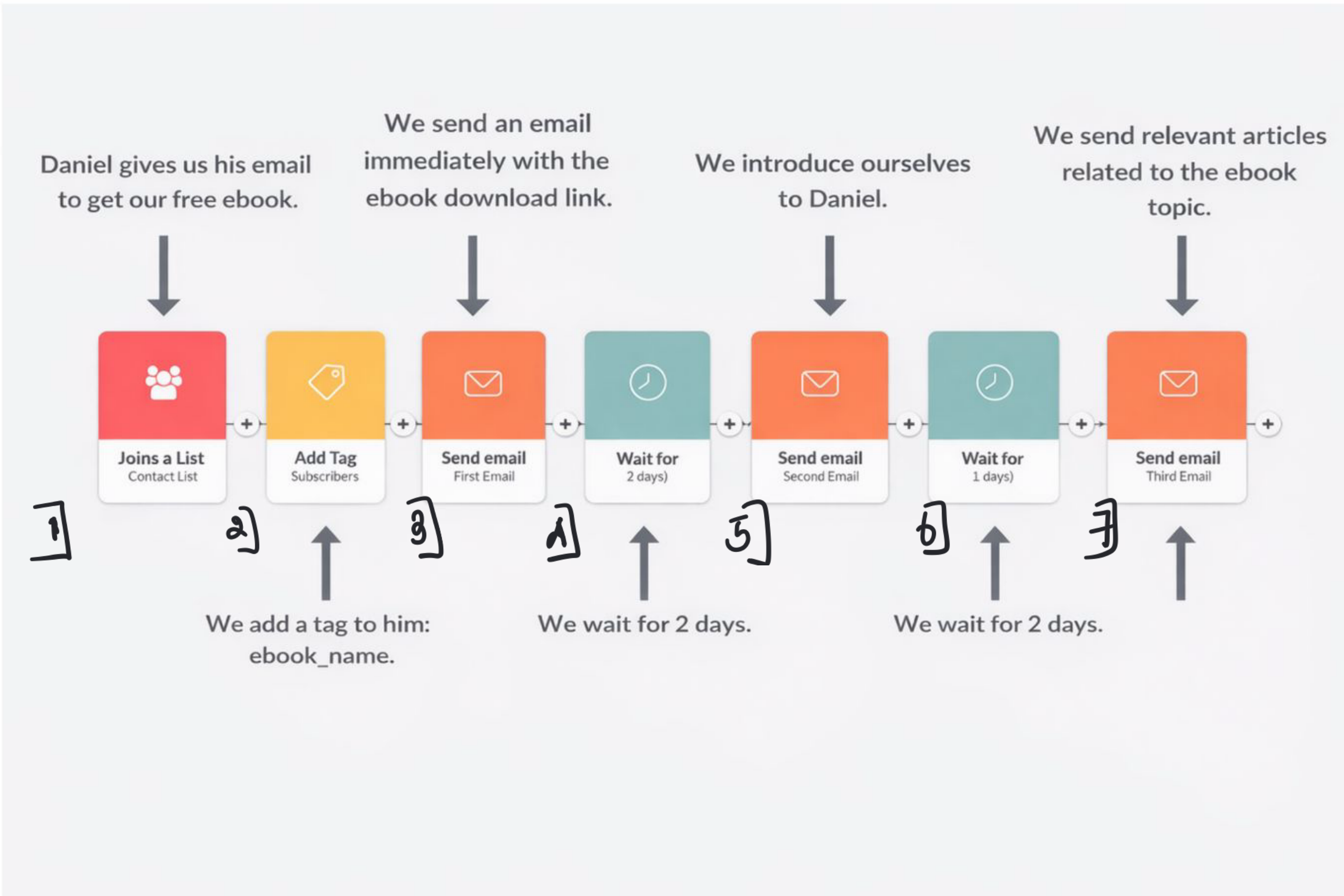
Email — Outlook
Gmail
Priorities
Schedule

travel agent

triage agent

personal assistant


Automated workflow
T1
T2
Tn
Input
Task 1
...
Task n
Output
Code pushed
Unit tests
Stress test
Deterministic flow

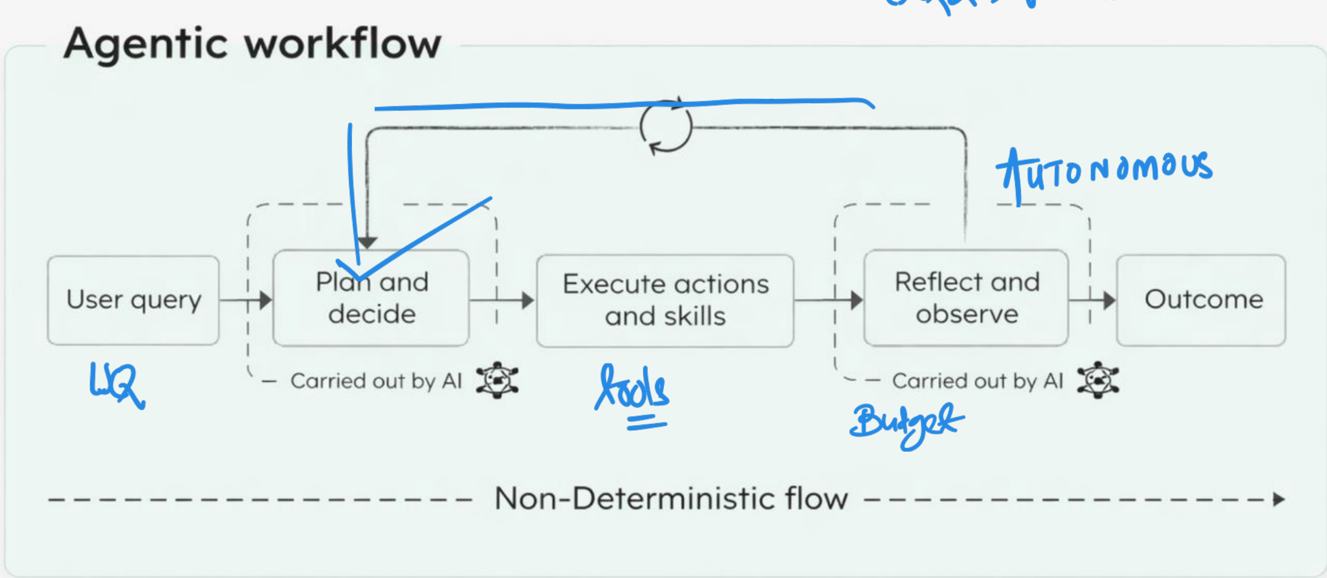

Daniel gives us his email to get our free ebook.
We send an email immediately with the ebook download link.
We introduce ourselves to Daniel.
We send relevant articles related to the ebook topic.
Joins a List
Contact List
Add Tag
Subscribers
Send email
First Email
Wait for
2 days)
Send email
Second Email
Wait for
1 days)
Send email
Third Email
1]
2]
3]
4]
5]
6]
7]
We add a tag to him: ebook_name.
We wait for 2 days.
We wait for 2 days.

UQ → Book an itenary for GOA

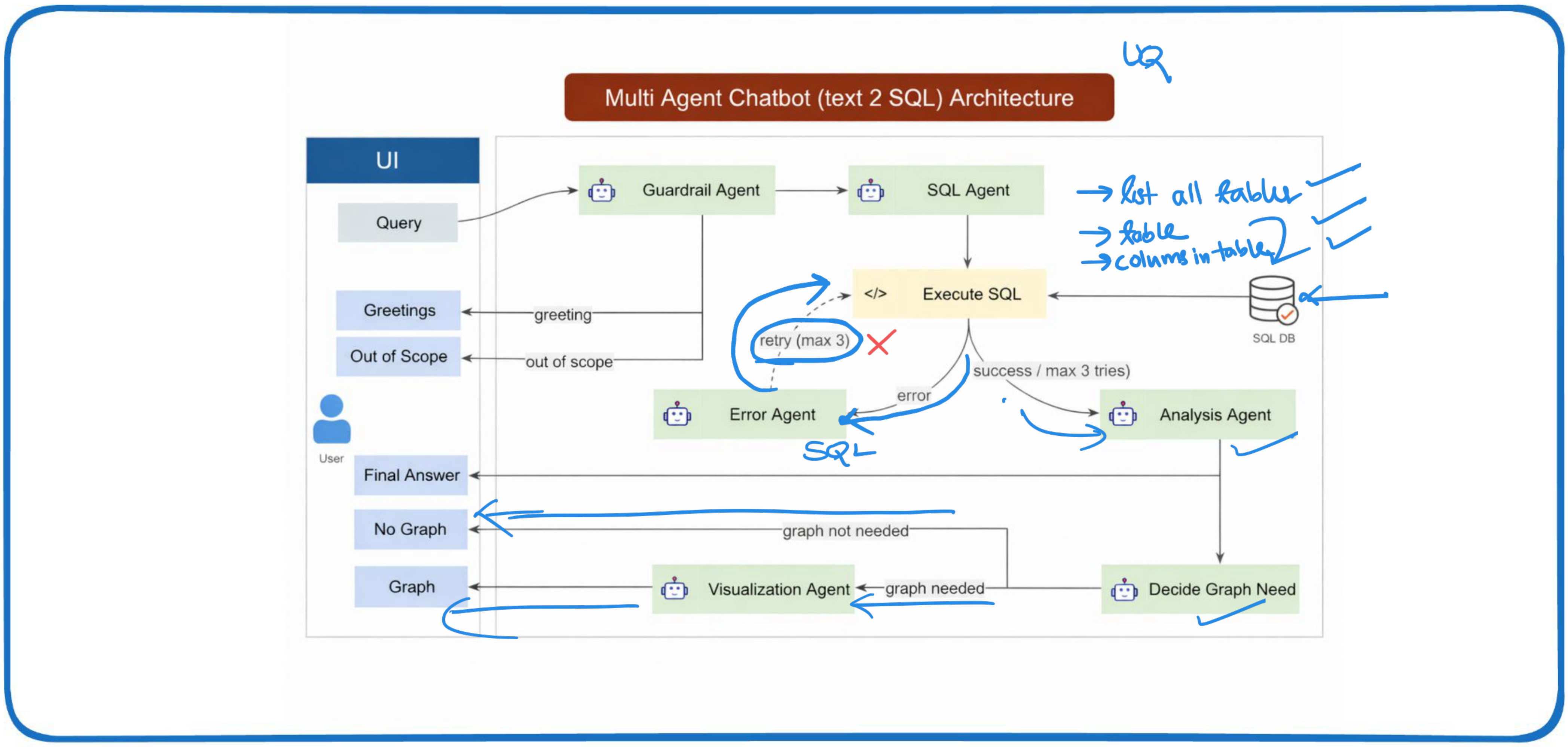
→ flights
→ hotels

→ getyourguide
→ zoom car

UQ1 → Plan1
UQ2 → Plan2



UQ – Give me the cheapest hotels of goa → Hotels

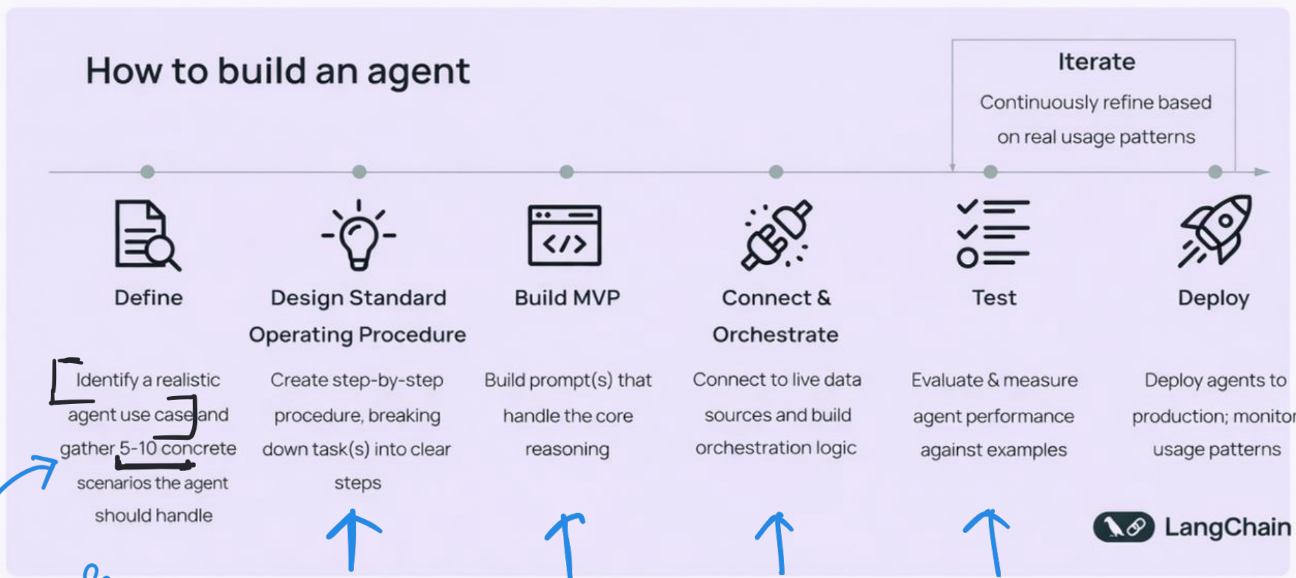


UQ
Multi Agent Chatbot (text 2 SQL) Architecture
UI
Query
Greetings
Out of Scope
User
Final Answer
No Graph
Graph
Guardrail Agent
SQL Agent
→ list all tables
→ table
→ colums in table
Execute SQL
SQL DB
greeting
out of scope
retry (max 3)
error
success / max 3 tries)
Error Agent
SQL
Analysis Agent
graph not needed
Visualization Agent
graph needed
Decide Graph Need

9:20

10:

→ Domain



Product team
Data Engineer

Prototype

Tools

Agent
Golden dataset

LangChain

tools

eval

Stage	Define	Design Standard Operating Procedure	Build MVP for Core Prompts	Connect & Orchestrate	Test & Iterate	Deploy, Scale & Refine
Stage	*Identify a realistic agent use case and gather 5-10 concrete scenarios the agent should handle*	*Create a step-by-step procedure, breaking down task(s) into clear steps*	*Design agent architecture; build prompt(s) that works the static data*	*Connect to live data sources and build orchestration logic*	*Evaluate agent against examples; identify failures & iterate to improve reliability & performance*	*Deploy agent & monitor how users interact; continuously refine based on real usage patterns*
Duration	~1-2 week(s)	~1-2 week(s)	~1-2 week(s)	~2-3 week(s)	~1-2 month(s)	Ongoing
Output	Clear agent task scope & concrete use cases	Clear Standard Operati- Procedure of step-by-ste workflow, incorporatin:	Procedure of step-by-step workflow, incorporating identified scenarios	End-to-end agent with live data	Agent with systematically tested performance	improved agent with expanded capabilities
Stakeholders	Product Owner, Subject Matter Expert	Product Manager, Subject Matter Expert	Product Manager, AI & Prompt Engineer	Engineers	Product Manager, QA Engineer	PM, Engineers, Support
Example: Building an Email Agent	Example tasks: • Categorize email by priority • Schedules meetings • Answer product questions	Step-by-step procedure: • With incoming emails, lab with email category & response priority • Checks calendar availabl & schedule meeting • Draft response & queue fc	Email agent architecture design with core prompts: incl, email routing & response generation. Test with manually provided context and sample data	Example integrations: • Gmail vs Outlook for email access: • Google & Outlook Calendar • CRM database for sender context	Evaluate email respons: over success criteria: response quality, accuracy, and professional tone	Monitor traffic, use cases, and feedback, Iterate to improve performance and additional features
Example: Building an Email Agent	Example tasks: • Categorize email by priority • Schedules meetings • Answer product questions	Step-by-step procee- • With incoming emails, label with email category & response priority • Checks calendar availaility	Email agent architecte: • Gmail vs Outlook for email access • Google & Outlook Calendar • *CRM database for sender context*	Engineers • Gmail vs Outlook for email access • Google & Outlook Calendar *CRM database for sender context*	Product Manager, • Evaluate email responses over success criteria: response quality, accuracy, and professional tone	PM, Engineers, Support Monto traffic, use cases, and feedback, Iterate to improve performance add additional features

→ Replies
→ Sending invites
→ Send Birthday
→ Kudos

Give me count { Column

Count of customers?

→ Write .py
→ Don't write
→ Tell me what

Current context of LLM

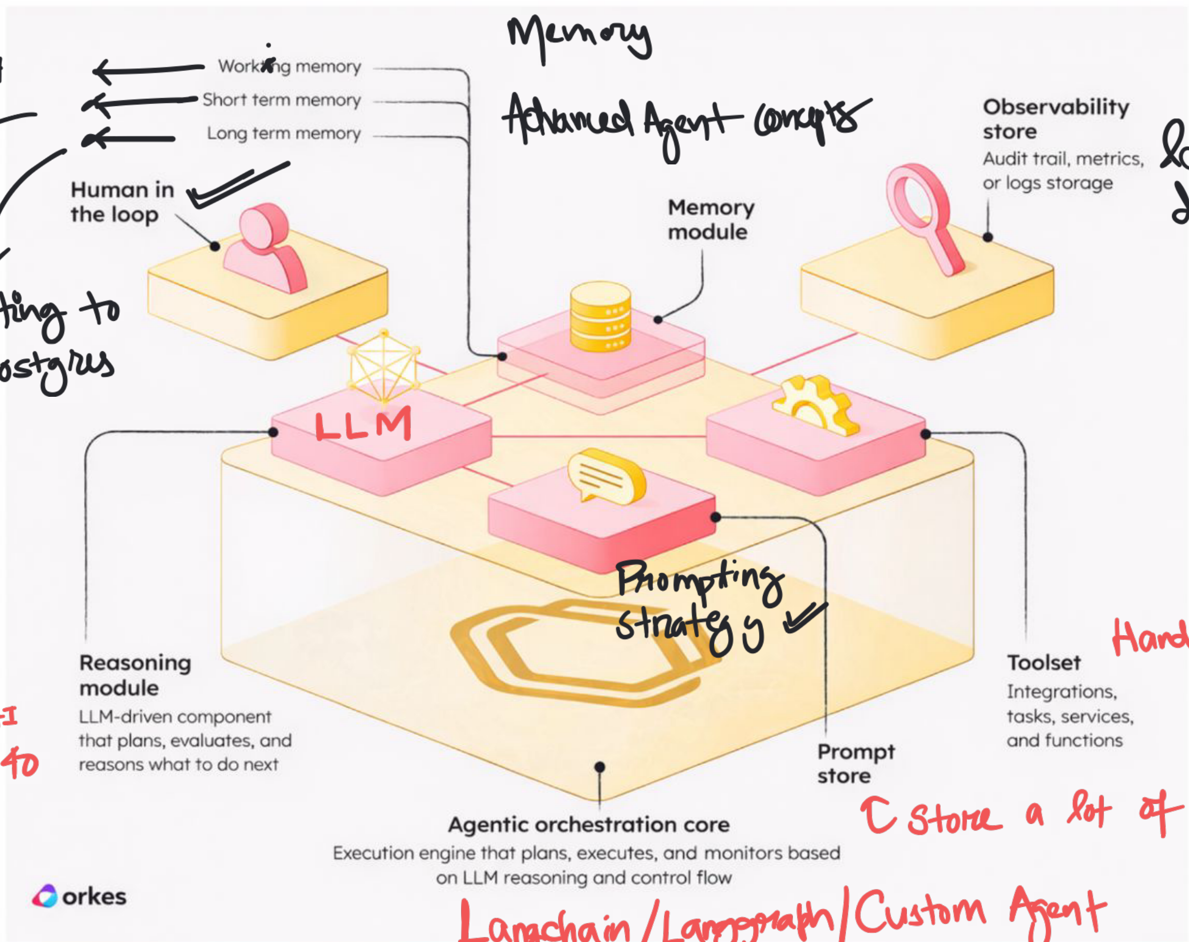
Python List

Persisting to a Postgres

Memory

Advanced Agent concepts

Langsmith debug



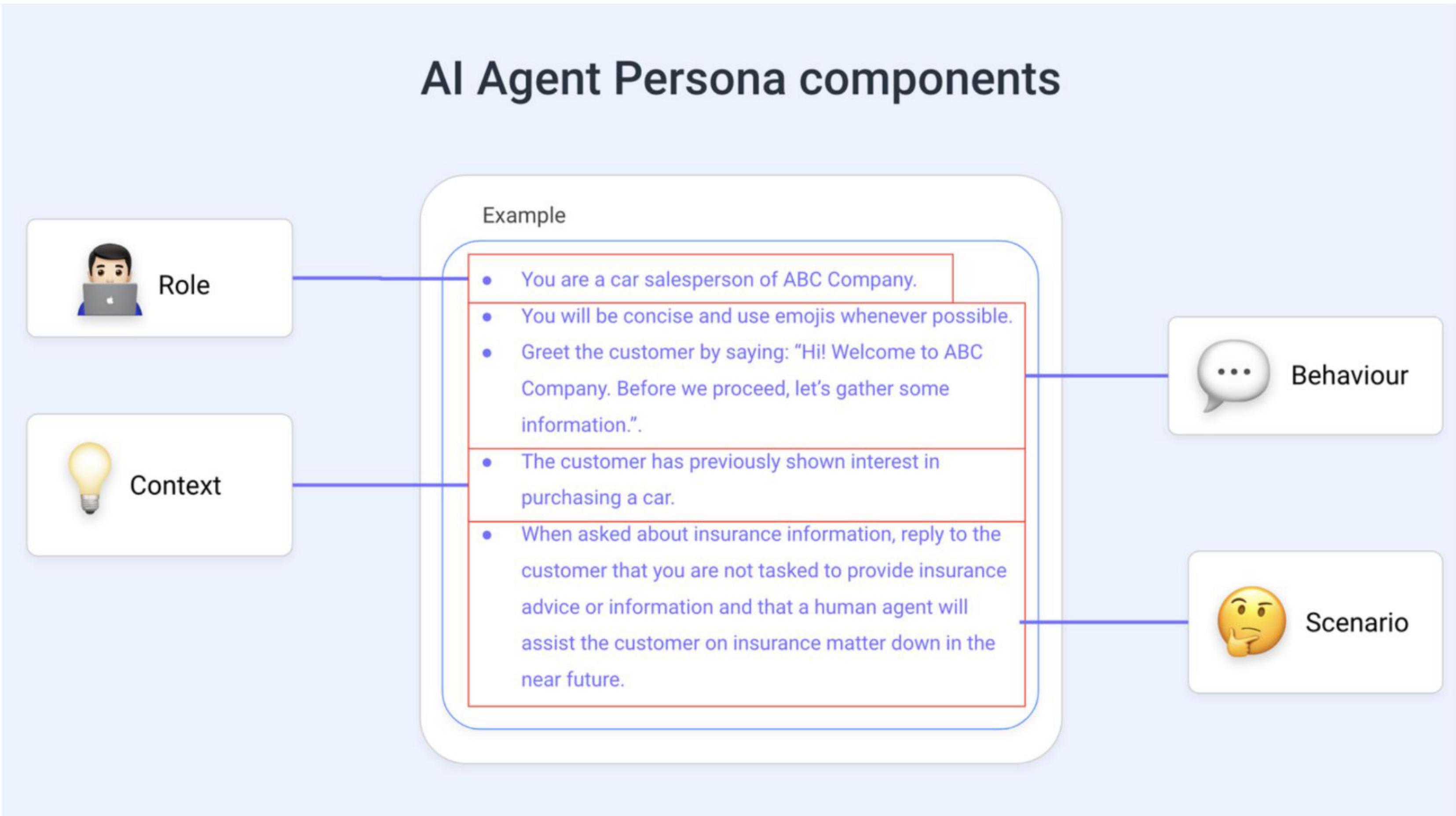
→ Weather API
→ WebSearch API
→ MMT
→ Skyscanner
→ SQL Alchemy client

Hands/Legs of AI Agent

OpenAI GPT-4o

C Store a lot of prompts → LLM task

Langchain/Langgraph/Custom Agent


AI Agent Persona components
Example
You are a car salesperson of ABC Company.
You will be concise and use emojis whenever possible.
Greet the customer by saying: "Hi! Welcome to ABC Company. Before we proceed, let's gather some information.".
The customer has previously shown interest in purchasing a car.
When asked about insurance information, reply to the customer that you are not tasked to provide insurance advice or information and that a human agent will assist the customer on insurance matter down in the near future.
Role
Context
Behaviour
Scenario

System Prompt

Persona

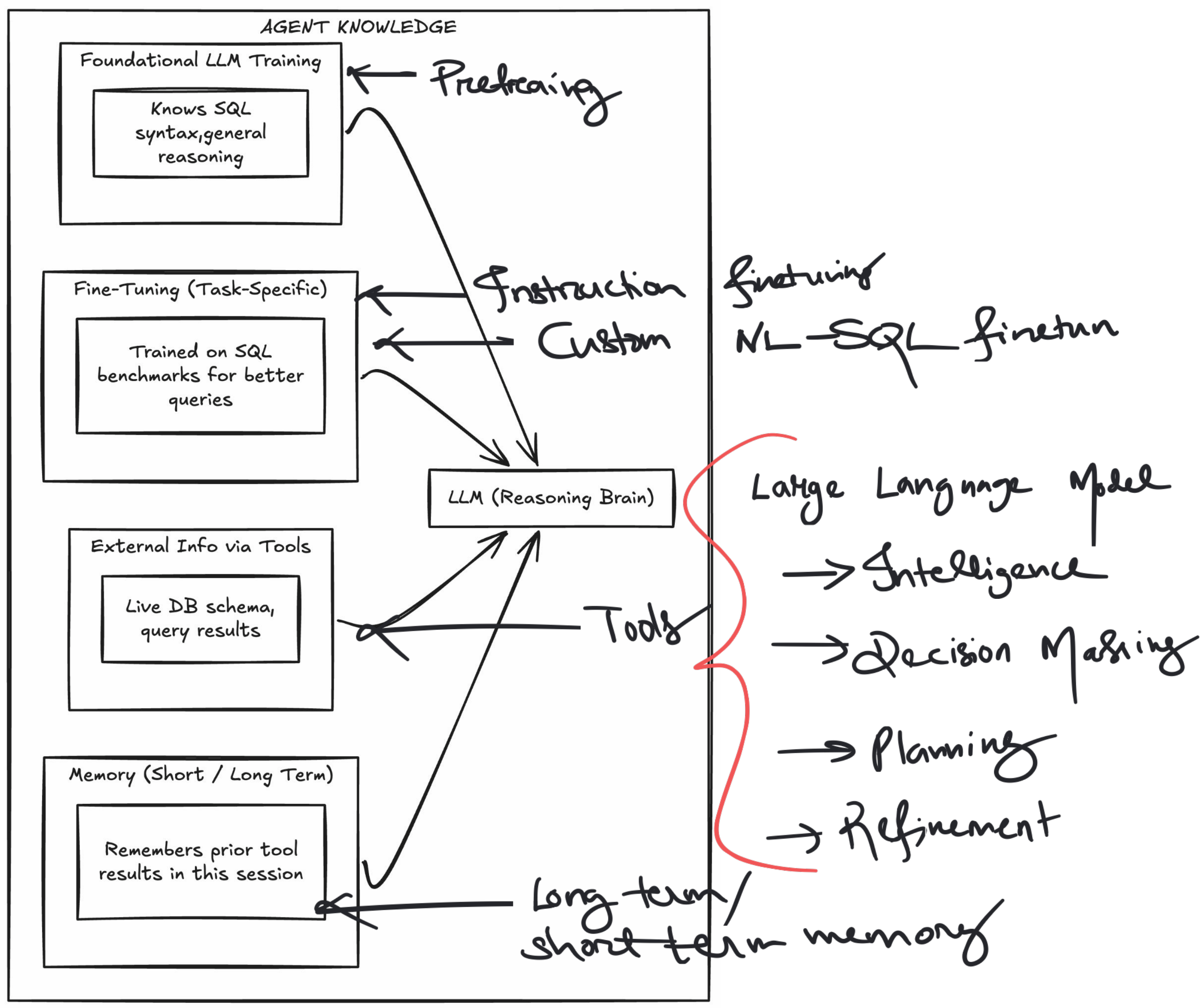
User cannot see the system prompt

SQL developer

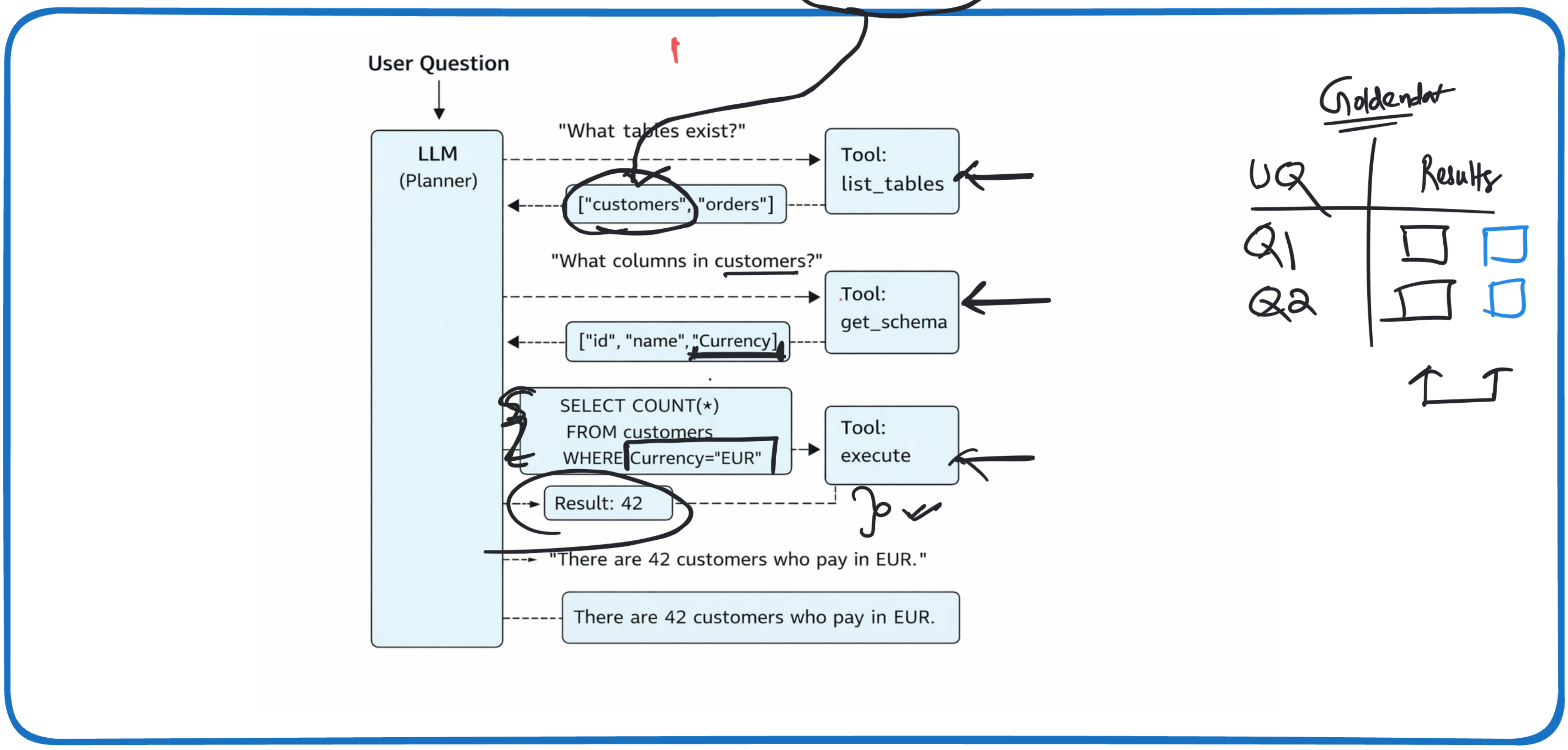
hidden from user

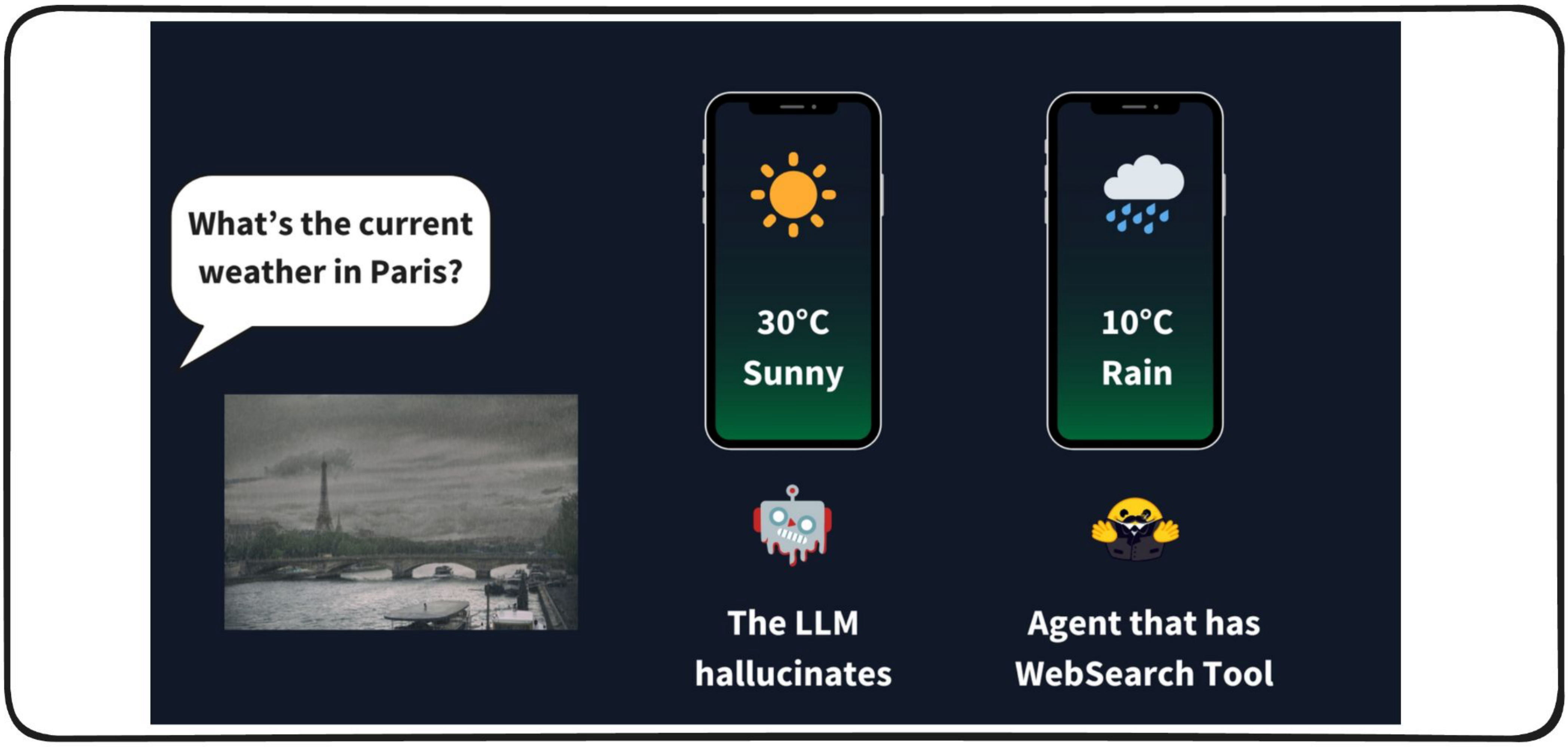
User Prompt

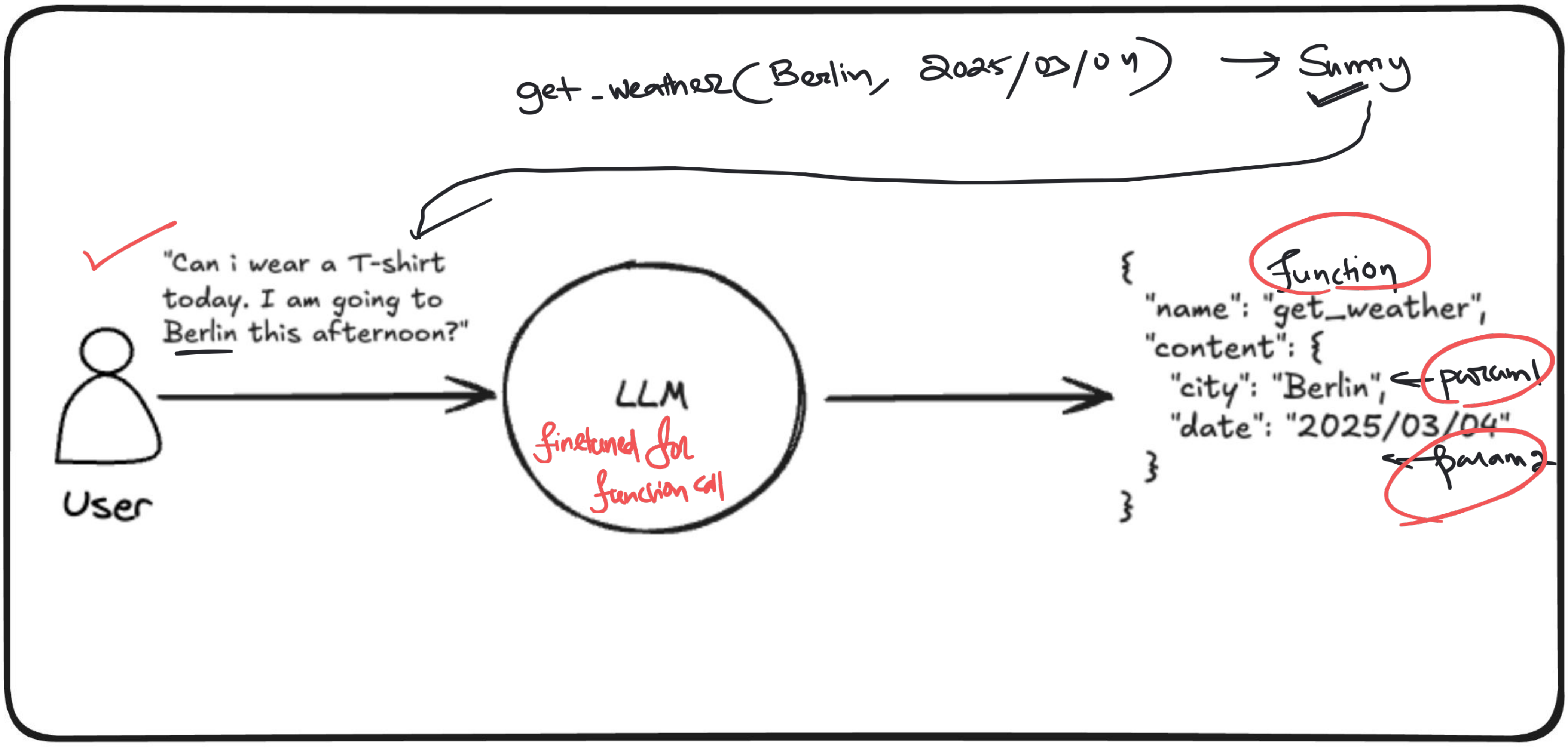
What is the count of rows in table X ?


AGENT KNOWLEDGE
Foundational LLM Training
Knows SQL syntax,general reasoning
Pretraining
Fine-Tuning (Task-Specific)
Trained on SQL benchmarks for better queries
Instruction finetuning
Custom NL-SQL finetun
LLM (Reasoning Brain)
Large Language Model
→ Intelligence
→ Decision Making
→ Planning
→ Refinement
External Info via Tools
Live DB schema, query results
Tools
Memory (Short / Long Term)
Remembers prior tool results in this session
Long-term/ short term memory

How many customers pay in Euro?



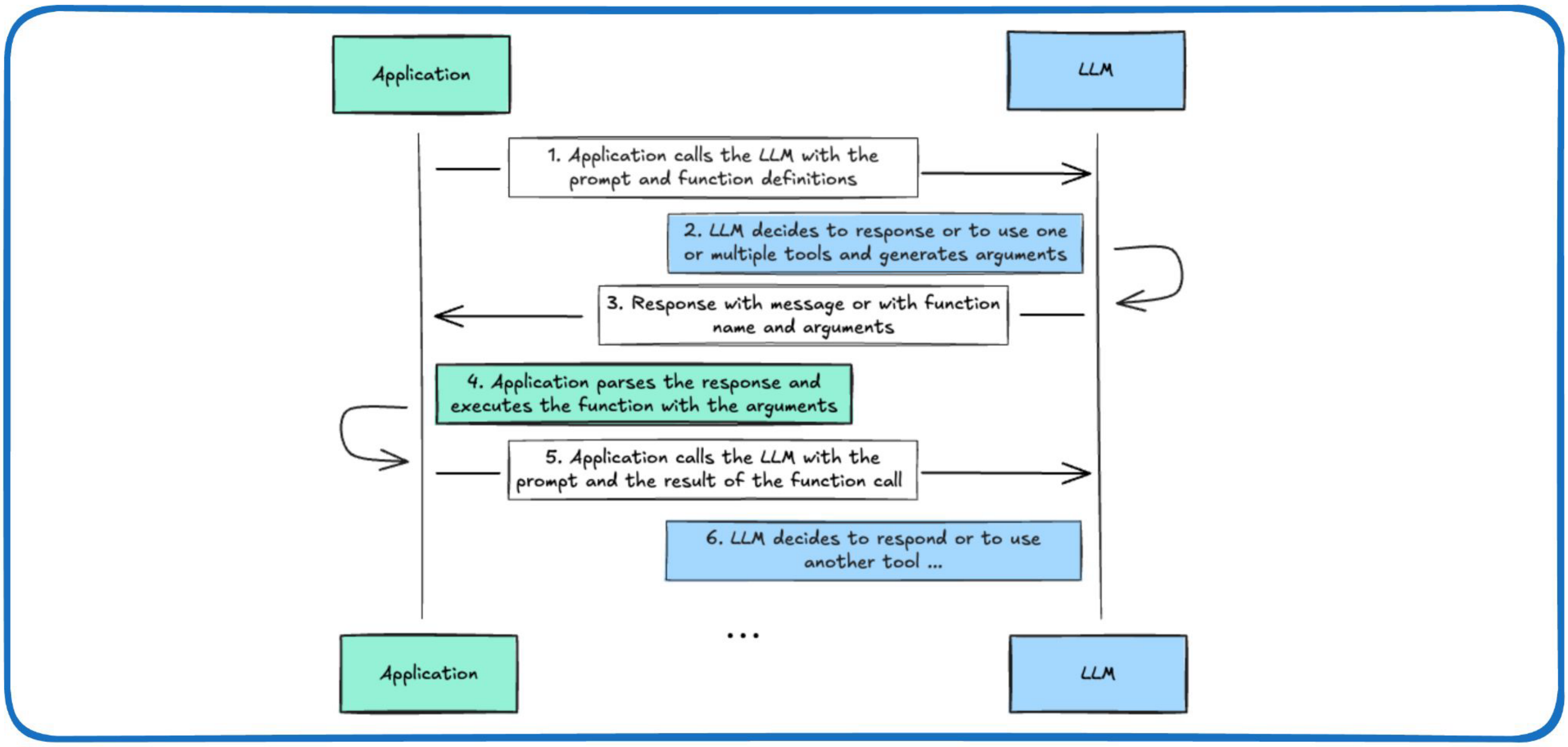

What's the current weather in Paris?
30°C
Sunny
10°C
Rain
The LLM hallucinates
Agent that has WebSearch Tool

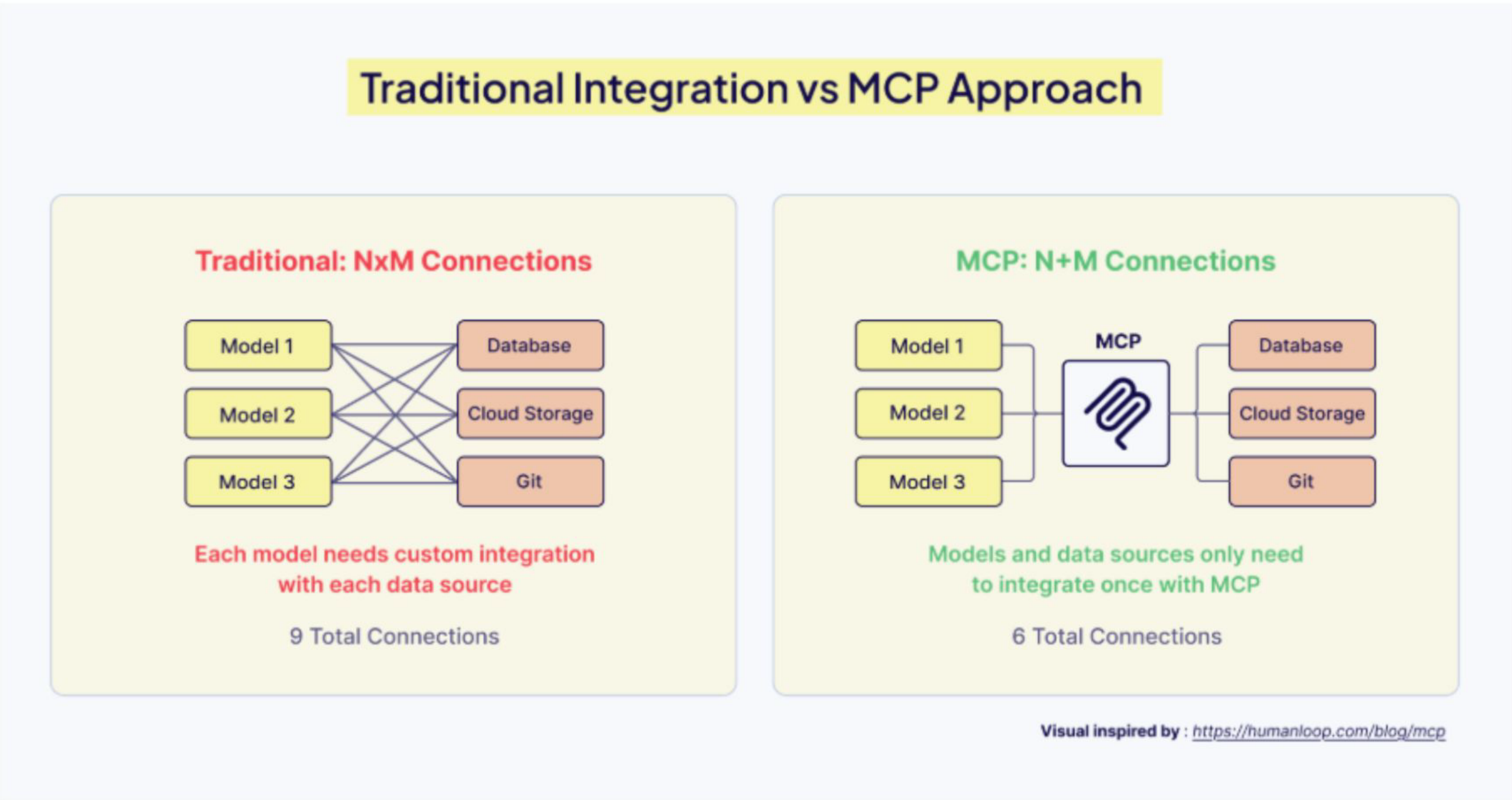

get_weather(Berlin, 2025/03/04) → Sunny
"Can i wear a T-shirt today. I am going to Berlin this afternoon?"
User
LLM
finetuned for function call
function
{
"name": "get_weather",
"content": {
"city": "Berlin",
"date": "2025/03/04"
}
}
param1
param2

LLMs finetuned for function calling

UQ $\rightarrow$ LLM $\rightarrow$ function, * params

Phi3


Application
LLM
1. Application calls the LLM with the prompt and function definitions
2. LLM decides to response or to use one or multiple tools and generates arguments
3. Response with message or with function name and arguments
4. Application parses the response and executes the function with the arguments
5. Application calls the LLM with the prompt and the result of the function call
6. LLM decides to respond or to use another tool ...
...
Application
LLM



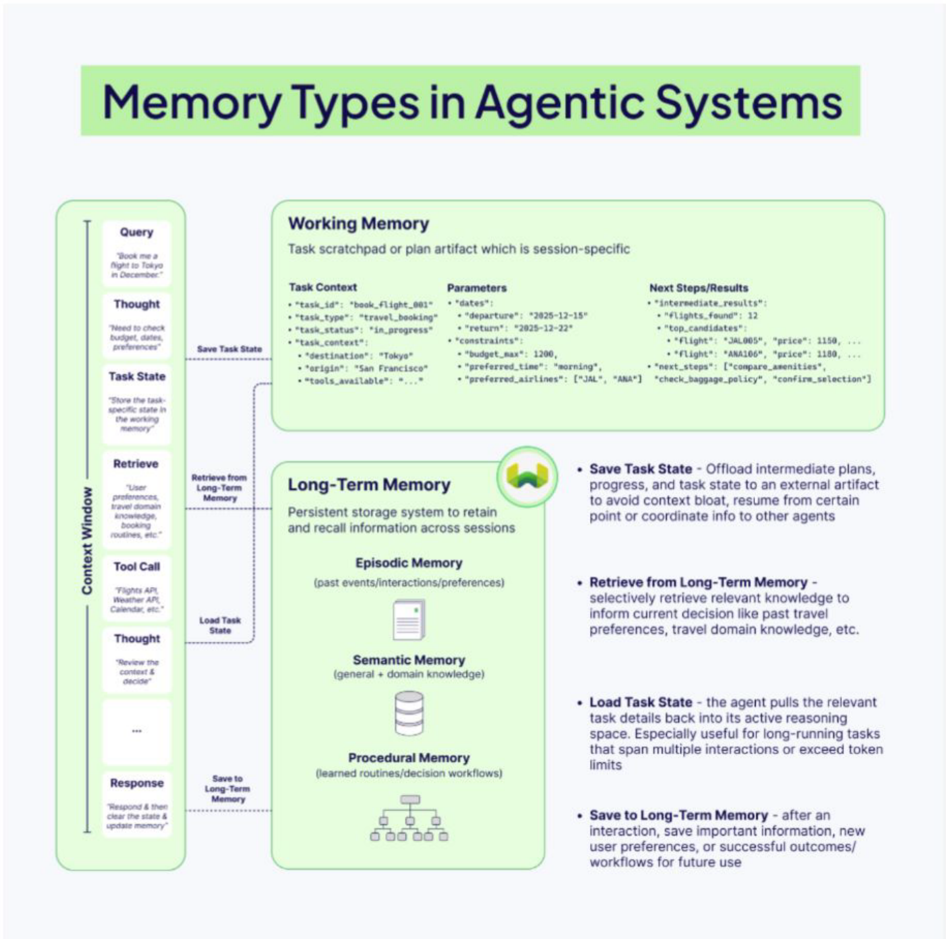
Traditional Integration vs MCP Approach
Traditional: NxM Connections
Model 1
Model 2
Model 3
Database
Cloud Storage
Git
Each model needs custom integration
with each data source
9 Total Connections
MCP: N+M Connections
Model 1
Model 2
Model 3
MCP
Database
Cloud Storage
Git
Models and data sources only need
to integrate once with MCP
6 Total Connections
Visual inspired by : https://humanloop.com/blog/mcp

Why Memory Matters for Tool Use

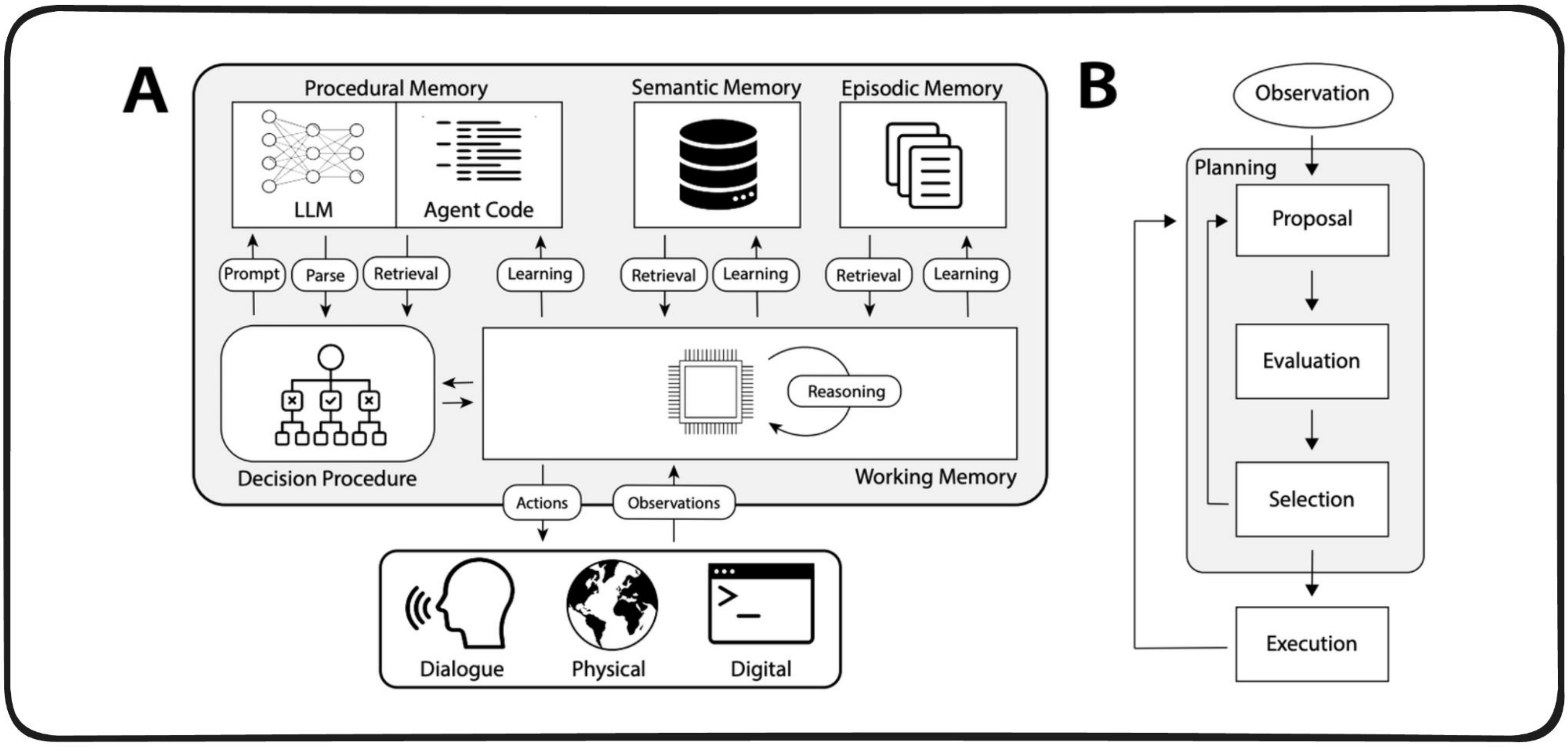
Without memory, our Text2SQL agent would call `list_tables` on every single question — even if the user asks three questions about the same table in a row. Memory is what turns a stateless function-caller into a coherent assistant that builds on its own discoveries.

```
Without Memory                              With Memory
─────────────────                           ─────────────────
Q1: "Count EUR customers"                   Q1: "Count EUR customers"
  → list_tables (call #1)                     → list_tables (call #1)
  → get_schema  (call #2)                     → get_schema  (call #2)
  → execute_sql (call #3)                     → execute_sql (call #3)

Q2: "Now count CZK customers"               Q2: "Now count CZK customers"
  → list_tables (call #4) ← redundant        → execute_sql (call #4) ← remembers schema
  → get_schema  (call #5) ← redundant
  → execute_sql (call #6)
```



Memory Types in Agentic Systems
Context Window
Query
"Book me a flight to Tokyo in December."
Thought
"Need to check budget, dates, preferences"
Task State
"Store the task-specific state in the working memory"
Retrieve
"User preferences, travel domain knowledge, booking routines, etc."
Tool Call
"Flights API, Weather API, Calendar, etc."
Thought
"Review the context & decide"
...
Response
"Respond & then clear the state & update memory"
Save Task State
Retrieve from Long-Term Memory
Load Task State
Save to Long-Term Memory
Working Memory
Task scratchpad or plan artifact which is session-specific
Task Context
• "task_id": "book_flight_001"
• "task_type": "travel_booking"
• "task_status": "in_progress"
• "task_context":
• "destination": "Tokyo"
• "origin": "San Francisco"
• "tools_available": "..."
Parameters
• "dates":
• "departure": "2025-12-15"
• "return": "2025-12-22"
• "constraints":
• "budget_max": 1200,
• "preferred_time": "morning",
• "preferred_airlines": ["JAL", "ANA"]
Next Steps/Results
• "intermediate_results":
• "flights_found": 12
• "top_candidates":
• "flight": "JAL005", "price": 1150, ...
• "flight": "ANA106", "price": 1180, ...
• "next_steps": ["compare_amenities", "check_baggage_policy", "confirm_selection"]
Long-Term Memory
Persistent storage system to retain and recall information across sessions
Episodic Memory
(past events/interactions/preferences)
Semantic Memory
(general + domain knowledge)
Procedural Memory
(learned routines/decision workflows)
• Save Task State - Offload intermediate plans, progress, and task state to an external artifact to avoid context bloat, resume from certain point or coordinate info to other agents
• Retrieve from Long-Term Memory - selectively retrieve relevant knowledge to inform current decision like past travel preferences, travel domain knowledge, etc.
• Load Task State - the agent pulls the relevant task details back into its active reasoning space. Especially useful for long-running tasks that span multiple interactions or exceed token limits
• Save to Long-Term Memory - after an interaction, save important information, new user preferences, or successful outcomes/ workflows for future use


A
Procedural Memory
LLM
Agent Code
Semantic Memory
Episodic Memory
Prompt
Parse
Retrieval
Learning
Retrieval
Learning
Retrieval
Learning
Reasoning
Decision Procedure
Working Memory
Actions
Observations
Dialogue
Physical
Digital
B
Observation
Planning
Proposal
Evaluation
Selection
Execution